EXERCICE 15 · CODE COMPLET

Migrer un écran vers View / Scene / Adapter

Tous les fichiers utilisés, en entier, dans l'ordre de lecture.

1. **correction** (76 fichiers)

1 correction

Styles fournis (copiés tels quels, pas tapés en live) : src/styles.css (488 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : .gitignore

package.json (gabarit Vite) : dépendances ajoutées : @reduxjs/toolkit ^2.11.2, @tanstack/react-query ^5.90.0, @tanstack/react-query-devtools ^5.90.0, axios ^1.13.0, react-redux ^9.2.0, react-router-dom ^7.11.0, @playwright/test ^1.56.0, vitest ^3.2.4

Images utilisées (non affichées) : public/icon.svg

Présents mais pas encore branchés (non affichés) : src/features/api-demo/screens/ApiDemoScreen/ApiDemoScreen.adapter.test.js, src/features/auth/screens/LoginScreen/LoginScreen.adapter.test.js, src/features/auth/utils/authLoader.test.js, src/features/cars/adapters/carAdapter.test.js, src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.adapter.test.js, src/features/cars/screens/HomeScreen/HomeScreen.adapter.test.js, src/features/cars/utils/colorUtils.test.js, src/features/errors/screens/ErrorScreen/ErrorScreen.adapter.test.js, src/features/favorites/favoritesSlice.test.js, src/features/favorites/screens/FavoritesScreen/FavoritesScreen.adapter.test.js

FICHIER **correction/src/main.jsx** 13 lignes

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import AppProviders from "./app/providers";
import App from "./app/App";
import "./styles.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <AppProviders>
      <App />
    </AppProviders>
  </StrictMode>
);
```

FICHIER **correction/src/app/providers.jsx** 17 lignes

```jsx
import { Provider } from "react-redux";
import { QueryClientProvider } from "@tanstack/react-query";
import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
import { queryClient } from "@/lib/queryClient";
import { store } from "./store";

// Tous les providers globaux au même endroit : Redux (favoris) + React Query (données API)
export default function AppProviders({ children }) {
  return (
    <Provider store={store}>
      <QueryClientProvider client={queryClient}>
        {children}
        <ReactQueryDevtools initialIsOpen={false} />
      </QueryClientProvider>
    </Provider>
  );
}
```

FICHIER **correction/src/app/App.jx** 6 lignes

```jsx
import { RouterProvider } from "react-router-dom";
import { router } from "./router";

export default function App() {
  return <RouterProvider router={router} />;
}
```

FICHIER **correction/src/app/store.js** 8 lignes

```jsx
import { configureStore } from "@reduxjs/toolkit";
import favoritesReducer from "@/features/favorites/favoritesSlice";

export const store = configureStore({
  reducer: {
    favorites: favoritesReducer,
  },
});
```

FICHIER **correction/src/app/router.jsx** 26 lignes

```jsx
import { createBrowserRouter } from "react-router-dom";
import RootLayout from "./RootLayout";
import { HomeScreen } from "@/features/cars/screens/HomeScreen";
import { CarDetailsScreen } from "@/features/cars/screens/CarDetailsScreen";
import { FavoritesScreen } from "@/features/favorites/screens/FavoritesScreen";
import { LoginScreen } from "@/features/auth/screens/LoginScreen";
import { ApiDemoScreen } from "@/features/api-demo/screens/ApiDemoScreen";
import { ErrorScreen } from "@/features/errors/screens/ErrorScreen";
import { NotFoundScreen } from "@/features/errors/screens/NotFoundScreen";
import { requireAuth, redirectIfAuthenticated } from "@/features/auth/utils/authLoader";

export const router = createBrowserRouter([
  { path: "/login", element: <LoginScreen />, loader: redirectIfAuthenticated },
  {
    path: "/",
    element: <RootLayout />,
    errorElement: <ErrorScreen />,
    children: [
      { index: true, element: <HomeScreen /> },
      { path: "favorites", element: <FavoritesScreen />, loader: requireAuth },
      { path: "car/:id", element: <CarDetailsScreen /> },
      { path: "api-demo", element: <ApiDemoScreen /> },
      { path: "*", element: <NotFoundScreen /> },
    ],
  },
]);
```

FICHIER **correction/src/app/RootLayout.jsx** 14 lignes

```jsx
import { Outlet } from "react-router-dom";
import { MainLayout } from "@/components/templates";
import { useCurrentUser } from "@/features/auth/hooks";

// Fait le lien entre la feature auth (l'utilisateur connecté) et le template, qui ne reçoit que des
props
export default function RootLayout() {
  const { user, logout } = useCurrentUser();

  return (
    <MainLayout userName={user?.userName || "Guest"} isLoggedIn={Boolean(user)} onLogout={logout}>
      <Outlet />
    </MainLayout>
  );
}
```

FICHIER **correction/src/features/favorites/favoritesSlice.js** 28 lignes

```js
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  favorites: [],
};

const favoritesSlice = createSlice({
  name: "favorites",
  initialState,
  reducers: {
    // Si la voiture est déjà en favori on la retire, sinon on l'ajoute
    toggleFavorite: (state, action) => {
      const carExists = state.favorites.some((car) => car.id === action.payload.id);
      if (carExists) {
        state.favorites = state.favorites.filter((car) => car.id !== action.payload.id);
      } else {
        state.favorites.push(action.payload);
      }
    },
  },
});

export const { toggleFavorite } = favoritesSlice.actions;

// Selectors
export const selectFavorites = (state) => state.favorites.favorites;

export default favoritesSlice.reducer;
```

FICHIER **correction/src/features/cars/screens/HomeScreen/index.js** 1 ligne

```js
export { HomeScreenView as HomeScreen } from "./HomeScreen.view";
```

FICHIER **correction/src/features/cars/screens/CarDetailsScreen/index.js** 1 ligne

```js
export { CarDetailsScreenView as CarDetailsScreen } from "./CarDetailsScreen.view";
```

FICHIER **correction/src/features/favorites/screens/FavoritesScreen/index.js** 1 ligne

```js
export { FavoritesScreenView as FavoritesScreen } from "./FavoritesScreen.view";
```

FICHIER **correction/src/features/auth/screens/LoginScreen/index.js** 1 ligne

```js
export { LoginScreenView as LoginScreen } from "./LoginScreen.view";
```

FICHIER **correction/src/features/api-demo/screens/ApiDemoScreen/index.js** 1 ligne

```js
export { ApiDemoScreenView as ApiDemoScreen } from "./ApiDemoScreen.view";
```

FICHIER **correction/src/features/errors/screens/ErrorScreen/index.js** 1 ligne

```js
export { ErrorScreenView as ErrorScreen } from "./ErrorScreen.view";
```

FICHIER **correction/src/features/errors/screens/NotFoundScreen/index.js** 1 ligne

```js
export { NotFoundScreenScene as NotFoundScreen } from "./NotFoundScreen.scene";
```

FICHIER **correction/src/features/auth/utils/authLoader.js** 19 lignes

```js
import { redirect } from "react-router-dom";
import { storage } from "./storage";

// Loader de /favorites : pas de token => redirection vers /login (avant même d'afficher la page)
export const requireAuth = () => {
  const token = storage.getToken();
```

```
  if (!token) {
    throw redirect("/login");
  }
  return { token };
};

// Loader de /login : déjà connecté => retour à l'accueil
export const redirectIfAuthenticated = () => {
  if (storage.getToken()) {
    throw redirect("/");
  }
  return null;
};
```

FICHIER **correction/src/components/templates/index.js** 3 lignes

```
export { MainLayout } from "./MainLayout";
export { Header } from "./Header";
export { Footer } from "./Footer";
```

FICHIER **correction/src/features/auth/hooks/index.js** 2 lignes

```
export { useCurrentUser } from "./useCurrentUser";
export { useLogin } from "./useLogin";
```

FICHIER **correction/src/features/cars/screens/HomeScreen/HomeScreen.view.jsx** 23 lignes

```
import { Loading, ErrorMessage } from "@/components/molecules";
import { useFavorites } from "@/features/favorites/hooks";
import { useCars } from "../../hooks";
import { adaptHomeCars, findCarById } from "./HomeScreen.adapter";
import { HomeScreenScene } from "./HomeScreen.scene";

const CATALOG_YEAR = 2012;

// Conteneur : récupère les données, gère chargement / erreur, adapte, puis passe la main à la Scene
export function HomeScreenView() {
  const { data, isLoading, error } = useCars(CATALOG_YEAR);
  const { favoriteIds, toggleFavorite } = useFavorites();

  if (isLoading) return <Loading text="Loading cars..." />;
  if (error) return <ErrorMessage message={`Error loading cars: ${error.message}`} />;

  const handleToggleFavorite = (id) => {
    const car = findCarById(data, id);
    if (car) toggleFavorite(car);
  };

  return <HomeScreenScene cars={adaptHomeCars(data, favoriteIds)} onToggleFavorite=
{handleToggleFavorite} />;
}
```

FICHIER **correction/src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.view.jsx** 28 lignes

```
import { useParams } from "react-router-dom";
import { Loading } from "@/components/molecules";
import { useFavorites } from "@/features/favorites/hooks";
import { useCarDetails } from "../../hooks";
import { adaptCarDetails } from "./CarDetailsScreen.adapter";
import { CarDetailsScreenScene } from "./CarDetailsScreen.scene";

export function CarDetailsScreenView() {
  const { id } = useParams();
  const { data, isLoading, error } = useCarDetails(id);
  const { favoriteIds, toggleFavorite } = useFavorites();
```

```
12
13    if (isLoading) return <Loading text="Loading car details..." />;
14
15    const car = adaptCarDetails(data, favoriteIds);
16
17    // Erreur réseau ou voiture absente : même message
18    if (error || !car) {
19      return (
20        <div className="container">
21          <p>Car not found</p>
22        </div>
23      );
24    }
25
26    // Dans Redux on stocke l'objet API brut, pas le modèle adapté
27    return <CarDetailsScreenScene car={car} onToggleFavorite={() => toggleFavorite(data.Car)} />;
28  }
```

FICHIER `correction/src/features/favorites/screens/FavoritesScreen/FavoritesScreen.view.jsx` — 15 lignes

```
1  import { useFavorites } from "../../hooks";
2  import { adaptFavoriteCars } from "./FavoritesScreen.adapter";
3  import { FavoritesScreenScene } from "./FavoritesScreen.scene";
4
5  // Pas d'appel API ici : les données viennent du store Redux
6  export function FavoritesScreenView() {
7    const { favorites, toggleFavorite } = useFavorites();
8
9    const handleToggleFavorite = (id) => {
10     const car = favorites.find((favorite) => favorite.id === id);
11     if (car) toggleFavorite(car);
12   };
13
14   return <FavoritesScreenScene cars={adaptFavoriteCars(favorites)} onToggleFavorite=
   {handleToggleFavorite} />;
15 }
```

FICHIER `correction/src/features/auth/screens/LoginScreen/LoginScreen.view.jsx` — 15 lignes

```
1  import { useLogin } from "../../hooks";
2  import { getLoginErrorMessage } from "./LoginScreen.adapter";
3  import { LoginScreenScene } from "./LoginScreen.scene";
4
5  export function LoginScreenView() {
6    const loginMutation = useLogin();
7
8    return (
9      <LoginScreenScene
10       onSubmit={(credentials) => loginMutation.mutate(credentials)}
11       isPending={loginMutation.isPending}
12       errorMessage={getLoginErrorMessage(loginMutation.error)}
13     />
14   );
15 }
```

FICHIER `correction/src/features/api-demo/screens/ApiDemoScreen/ApiDemoScreen.view.jsx` — 28 lignes

```
1  import { useState } from "react";
2  import { adaptDemoResult } from "./ApiDemoScreen.adapter";
3  import { ApiDemoScreenScene } from "./ApiDemoScreen.scene";
4
5  const SIMULATED_DELAY_MS = 2000;
6
7  // Une page pour voir les 3 états d'un appel API : loading, succès, erreur
8  export function ApiDemoScreenView() {
```

```
  const [isLoading, setIsLoading] = useState(false);
  const [result, setResult] = useState(null);
  const [crash, setCrash] = useState(false);

  // Pour tester l'Error Boundary : une erreur pendant le render
  if (crash) {
    throw new Error("Crash volontaire pour tester l'Error Boundary");
  }

  const simulate = (ok) => {
    setResult(null);
    setIsLoading(true);
    setTimeout(() => {
      setIsLoading(false);
      setResult(adaptDemoResult(ok));
    }, SIMULATED_DELAY_MS);
  };

  return <ApiDemoScreenScene isLoading={isLoading} result={result} onSimulate={simulate} onCrash={() => setCrash(true)} />;
}
```

FICHIER `correction/src/features/errors/screens/ErrorScreen/ErrorScreen.view.jsx` — 14 lignes

```
import { useNavigate, useRouteError } from "react-router-dom";
import { getErrorMessage } from "./ErrorScreen.adapter";
import { ErrorScreenScene } from "./ErrorScreen.scene";

// errorElement de React Router : une erreur dans une page affiche cet écran au lieu de casser l'application
export function ErrorScreenView() {
  const error = useRouteError();
  const navigate = useNavigate();

  console.error("ErrorBoundary:", error);

  // navigate(0) recharge la page courante
  return <ErrorScreenScene message={getErrorMessage(error)} onRetry={() => navigate(0)} />;
}
```

FICHIER `correction/src/features/errors/screens/NotFoundScreen/NotFoundScreen.scene.jsx` — 12 lignes

```
import { ButtonLink } from "@/components/atoms";

// Page 404 : aucune donnée, donc pas de View ni d'adapter, la Scene suffit
export function NotFoundScreenScene() {
  return (
    <div className="center-page">
      <h1 className="not-found-title">404</h1>
      <p className="not-found-text">Page not found</p>
      <ButtonLink to="/">Go Home</ButtonLink>
    </div>
  );
}
```

FICHIER `correction/src/features/auth/utils/storage.js` — 19 lignes

```
// Le token et l'utilisateur sont gardés dans le localStorage (la session survit au rechargement)
const TOKEN_KEY = "auth_token";
const USER_KEY = "auth_user";

// Événement envoyé après un login pour prévenir le Header (dans le même onglet)
export const USER_UPDATED_EVENT = "userUpdated";

export const storage = {
```

```
  setToken: (token) => localStorage.setItem(TOKEN_KEY, token),
  getToken: () => localStorage.getItem(TOKEN_KEY),
  removeToken: () => localStorage.removeItem(TOKEN_KEY),

  setUser: (user) => localStorage.setItem(USER_KEY, JSON.stringify(user)),
  getUser: () => {
    const user = localStorage.getItem(USER_KEY);
    return user ? JSON.parse(user) : null;
  },
  removeUser: () => localStorage.removeItem(USER_KEY),
};
```

FICHIER **correction/src/components/templates/MainLayout.jsx** 13 lignes

```
import { Header } from "./Header";
import { Footer } from "./Footer";

// La mise en page commune : en-tête, contenu de la page (children), pied de page
export function MainLayout({ userName, isLoggedIn, onLogout, children }) {
  return (
    <div className="app-layout">
      <Header userName={userName} isLoggedIn={isLoggedIn} onLogout={onLogout} />
      <main className="main-content">{children}</main>
      <Footer />
    </div>
  );
}
```

FICHIER **correction/src/components/templates/Header.jsx** 38 lignes

```
import { Link } from "react-router-dom";
import { Button, ButtonLink } from "../atoms";

// L'en-tête : il ne connaît pas le stockage, il reçoit le nom affiché et l'action de déconnexion
export function Header({ userName, isLoggedIn, onLogout }) {
  return (
    <header className="header">
      <div className="header-content">
        <Link to="/" className="logo">
          Car Catalog
        </Link>
        <nav className="nav">
          <Link to="/" className="nav-link">
            Home
          </Link>
          <Link to="/favorites" className="nav-link">
            Favorites
          </Link>
          <Link to="/api-demo" className="nav-link">
            API Demo
          </Link>
        </nav>
        <div className="user-section">
          <span className="username">{userName}</span>
          {isLoggedIn ? (
            <Button size="sm" onClick={onLogout}>
              Logout
            </Button>
          ) : (
            <ButtonLink to="/login" size="sm">
              Login
            </ButtonLink>
          )}
        </div>
      </div>
```

```
    </header>
  );
}
```

FICHIER `correction/src/components/templates/Footer.jsx` — 7 lignes

```jsx
export function Footer() {
  return (
    <footer className="footer">
      <p>&copy; 2026 Car Catalog. All rights reserved.</p>
    </footer>
  );
}
```

FICHIER `correction/src/features/auth/hooks/useCurrentUser.js` — 29 lignes

```js
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { storage, USER_UPDATED_EVENT } from "../utils/storage";

// L'utilisateur connecté (ou null) et la déconnexion
export const useCurrentUser = () => {
  const navigate = useNavigate();
  const [user, setUser] = useState(storage.getUser);

  // On écoute les changements de l'utilisateur (login dans cet onglet ou dans un autre)
  useEffect(() => {
    const refreshUser = () => setUser(storage.getUser());
    window.addEventListener("storage", refreshUser);
    window.addEventListener(USER_UPDATED_EVENT, refreshUser);
    return () => {
      window.removeEventListener("storage", refreshUser);
      window.removeEventListener(USER_UPDATED_EVENT, refreshUser);
    };
  }, []);

  const logout = () => {
    storage.removeToken();
    storage.removeUser();
    setUser(null);
    navigate("/login");
  };

  return { user, logout };
};
```

FICHIER `correction/src/features/auth/hooks/useLogin.js` — 22 lignes

```js
import { useMutation } from "@tanstack/react-query";
import { useNavigate } from "react-router-dom";
import { login } from "../api/authApi";
import { storage, USER_UPDATED_EVENT } from "../utils/storage";

// La mutation de login : enregistre la session puis retourne à l'accueil
export const useLogin = () => {
  const navigate = useNavigate();

  return useMutation({
    mutationFn: ({ userName, password }) => login(userName, password),
    onSuccess: (response) => {
      if (response.session) storage.setToken(response.session);
      if (response.userDetails) {
        storage.setUser(response.userDetails);
        // On prévient le Header que l'utilisateur a changé
        window.dispatchEvent(new CustomEvent(USER_UPDATED_EVENT));
```

```
      }
      navigate("/");
    },
  });
};
```

FICHIER **correction/src/components/molecules/index.js** 7 lignes

```
export { FavoriteButton } from "./FavoriteButton";
export { InfoRow } from "./InfoRow";
export { DetailItem } from "./DetailItem";
export { FormField } from "./FormField";
export { Loading } from "./Loading";
export { ErrorMessage } from "./ErrorMessage";
export { StatusMessage } from "./StatusMessage";
```

FICHIER **correction/src/features/favorites/hooks/index.js** 1 ligne

```
export { useFavorites } from "./useFavorites";
```

FICHIER **correction/src/features/cars/hooks/index.js** 2 lignes

```
export { useCars } from "./useCars";
export { useCarDetails } from "./useCarDetails";
```

FICHIER **correction/src/features/cars/screens/HomeScreen/HomeScreen.adapter.js** 15 lignes

```
import { adaptCarCard } from "../../adapters/carAdapter";

// La page d'accueil n'affiche que les 6 premières voitures
export const HOME_CARS_LIMIT = 6;

const getCarsList = (response) => (Array.isArray(response?.Cars) ? response.Cars : []);

// Réponse de l'API { Cars: [...] } -> liste de cartes prêtes à afficher
export const adaptHomeCars = (response, favoriteIds = []) =>
  getCarsList(response)
    .slice(0, HOME_CARS_LIMIT)
    .map((dto) => adaptCarCard(dto, favoriteIds.includes(dto.id)));

// Retrouve l'objet API brut d'une voiture (c'est lui qu'on stocke dans les favoris)
export const findCarById = (response, id) => getCarsList(response).find((car) => car.id === id);
```

FICHIER **correction/src/features/cars/screens/HomeScreen/HomeScreen.scene.jsx** 11 lignes

```
import { CarGrid } from "@/components/organisms";

// Présentation pure : reçoit les cartes déjà adaptées
export function HomeScreenScene({ cars, onToggleFavorite }) {
  return (
    <div className="container">
      <h1 className="page-title">Car Catalog</h1>
      <CarGrid cars={cars} onToggleFavorite={onToggleFavorite} />
    </div>
  );
}
```

FICHIER **correction/src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.adapter.js** 14 lignes

```
import { adaptCarBase, orMissing } from "../../adapters/carAdapter";

// Réponse de l'API { Car: {...} } -> modèle de la page détails (null si la voiture est absente)
export const adaptCarDetails = (response, favoriteIds = []) => {
  const dto = response?.Car;
  if (!dto) return null;

```

```
  return {
    ...adaptCarBase(dto),
    brand: orMissing(dto.car),
    model: orMissing(dto.car_model),
    isFavorite: favoriteIds.includes(dto.id),
  };
};
```

FICHIER `correction/src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.scene.jsx`

49 lignes

```
import { Link } from "react-router-dom";
import { BackIcon, ColorDot, Price, StatusBadge } from "@/components/atoms";
import { DetailItem, FavoriteButton } from "@/components/molecules";

// Présentation pure de la fiche voiture
export function CarDetailsScreenScene({ car, onToggleFavorite }) {
  return (
    <div className="container">
      <Link to="/" className="back-link">
        <BackIcon />
        Back to Catalog
      </Link>

      <div className="card detail-card">
        <div className="detail-header">
          <h1 className="detail-title">{car.title}</h1>
          <FavoriteButton isActive={car.isFavorite} onToggle={onToggleFavorite} />
        </div>

        <div className="detail-grid">
          <DetailItem label="Brand">
            <span className="detail-value">{car.brand}</span>
          </DetailItem>
          <DetailItem label="Model">
            <span className="detail-value">{car.model}</span>
          </DetailItem>
          <DetailItem label="Color">
            <div className="color-detail">
              <ColorDot color={car.colorHex} />
              <span className="detail-value">{car.colorName}</span>
            </div>
          </DetailItem>
          <DetailItem label="Year">
            <span className="detail-value">{car.year}</span>
          </DetailItem>
          <DetailItem label="VIN">
            <span className="detail-value">{car.vin}</span>
          </DetailItem>
          <DetailItem label="Price">
            <Price value={car.priceLabel} className="detail-value" />
          </DetailItem>
          <DetailItem label="Availability">
            <StatusBadge isAvailable={car.isAvailable} label={car.statusLabel} className="detail-
value" />
          </DetailItem>
        </div>
      </div>
    </div>
  );
}
```

FICHIER `correction/src/features/favorites/screens/FavoritesScreen/FavoritesScreen.adapter.js`

4 lignes

```
import { adaptCarCard } from "@/features/cars/adapters/carAdapter";
```

```
2
3  // Les favoris (objets API bruts stockés dans Redux) -> cartes prêtes à afficher, toutes en favori
4  export const adaptFavoriteCars = (favorites = []) => favorites.map((dto) => adaptCarCard(dto,
   true));
```

FICHIER `correction/src/features/favorites/screens/FavoritesScreen/FavoritesScreen.scene.jsx` — 14 lignes

```jsx
1  import { CarGrid } from "@/components/organisms";
2
3  export function FavoritesScreenScene({ cars, onToggleFavorite }) {
4    return (
5      <div className="container">
6        <h1 className="page-title">My Favorites</h1>
7        {cars.length === 0 ? (
8          <p className="subtitle">No favorites yet. Add some cars from the home page!</p>
9        ) : (
10         <CarGrid cars={cars} onToggleFavorite={onToggleFavorite} />
11       )}
12     </div>
13   );
14 }
```

FICHIER `correction/src/features/auth/screens/LoginScreen/LoginScreen.adapter.js` — 7 lignes

```js
1  export const DEFAULT_LOGIN_ERROR = "Login failed. Please try again.";
2
3  // Erreur axios -> message lisible : message du serveur, sinon message technique, sinon message par
   défaut
4  export const getLoginErrorMessage = (error) => {
5    if (!error) return null;
6    return error.response?.data?.message || error.message || DEFAULT_LOGIN_ERROR;
7  };
```

FICHIER `correction/src/features/auth/screens/LoginScreen/LoginScreen.scene.jsx` — 13 lignes

```jsx
1  import { LoginForm } from "@/components/organisms";
2
3  export function LoginScreenScene({ onSubmit, isPending, errorMessage }) {
4    return (
5      <div className="login-container">
6        <div className="card login-card">
7          <h1 className="login-title">Welcome Back</h1>
8          <p className="subtitle">Sign in to your account</p>
9          <LoginForm onSubmit={onSubmit} isPending={isPending} errorMessage={errorMessage} />
10       </div>
11     </div>
12   );
13 }
```

FICHIER `correction/src/features/api-demo/screens/ApiDemoScreen/ApiDemoScreen.adapter.js` — 5 lignes

```js
1  // Résultat simulé (succès ou échec) -> encadré à afficher
2  export const adaptDemoResult = (ok) =>
3    ok
4      ? { variant: "success", message: "Data loaded successfully!" }
5      : { variant: "error", message: "Failed to load data. Please try again." };
```

FICHIER `correction/src/features/api-demo/screens/ApiDemoScreen/ApiDemoScreen.scene.jsx` — 31 lignes

```jsx
1  import { Button } from "@/components/atoms";
2  import { StatusMessage } from "@/components/molecules";
3
4  export function ApiDemoScreenScene({ isLoading, result, onSimulate, onCrash }) {
5    return (
6      <div className="container">
```

```
      <h1 className="page-title">API Demo</h1>

      <div className="card api-demo-card">
        <p className="subtitle">Test loading states and error messages</p>

        <div className="button-group">
          <Button variant="success" onClick={() => onSimulate(true)} disabled={isLoading}>
            Simulate Success
          </Button>
          <Button variant="danger" onClick={() => onSimulate(false)} disabled={isLoading}>
            Simulate Error
          </Button>
        </div>

        <div className="button-group">
          <Button onClick={onCrash}>Crash the page (Error Boundary)</Button>
        </div>

        {isLoading && <StatusMessage variant="loading">Loading...</StatusMessage>}

        {result && <StatusMessage variant={result.variant}>{result.message}</StatusMessage>}
      </div>
    </div>
  );
}
```

FICHIER `correction/src/features/errors/screens/ErrorScreen/ErrorScreen.adapter.js` — 4 lignes

```
export const DEFAULT_ERROR_MESSAGE = "An unexpected error occurred";

// L'erreur levée par une route (Error, Response, valeur quelconque) -> message à afficher
export const getErrorMessage = (error) => error?.message || DEFAULT_ERROR_MESSAGE;
```

FICHIER `correction/src/features/errors/screens/ErrorScreen/ErrorScreen.scene.jsx` — 15 lignes

```
import { AlertIcon, Button, ButtonLink } from "@/components/atoms";

export function ErrorScreenScene({ message, onRetry }) {
  return (
    <div className="center-page error-content">
      <AlertIcon size={64} />
      <h1 className="error-title">Something went wrong!</h1>
      <p className="muted">{message}</p>
      <div className="button-group">
        <Button onClick={onRetry}>Try again</Button>
        <ButtonLink to="/">Go Home</ButtonLink>
      </div>
    </div>
  );
}
```

FICHIER `correction/src/components/atoms/index.js` — 6 lignes

```
export { HeartIcon, AlertIcon, CheckIcon, BackIcon } from "./Icons";
export { Button, ButtonLink } from "./Button";
export { ColorDot } from "./ColorDot";
export { StatusBadge } from "./StatusBadge";
export { Price } from "./Price";
export { Spinner } from "./Spinner";
```

FICHIER `correction/src/features/auth/api/authApi.js` — 5 lignes

```
import axiosQuery from "@/lib/axiosQuery";

// Login : l'API renvoie { session (le token), userDetails }
export const login = (userName, password) =>
```

```
  axiosQuery("post", "/api/login", { userName, password, verified: true, blocked: false, delay: 0
});
```

FICHIER `correction/src/components/molecules/FavoriteButton.jsx` — 10 lignes

```jsx
import { HeartIcon } from "../atoms";

// Le bouton cœur : plein et rouge quand la voiture est en favori
export function FavoriteButton({ isActive, onToggle }) {
  return (
    <button className={`favorite-btn ${isActive ? "favorite-active" : ""}`} onClick={onToggle}>
      <HeartIcon filled={isActive} />
    </button>
  );
}
```

FICHIER `correction/src/components/molecules/InfoRow.jsx` — 9 lignes

```jsx
// Une ligne "Label: valeur" de la carte voiture
export function InfoRow({ label, children }) {
  return (
    <div className="info-row">
      <span className="label">{`${label}:`}</span>
      {children}
    </div>
  );
}
```

FICHIER `correction/src/components/molecules/DetailItem.jsx` — 9 lignes

```jsx
// Une case de la grille de la page détails : petit label au-dessus, valeur en dessous
export function DetailItem({ label, children }) {
  return (
    <div className="detail-item">
      <span className="detail-label">{label}</span>
      {children}
    </div>
  );
}
```

FICHIER `correction/src/components/molecules/FormField.jsx` — 11 lignes

```jsx
// Un champ de formulaire avec son label
export function FormField({ id, label, type = "text", placeholder, required = false }) {
  return (
    <div className="form-group">
      <label htmlFor={id} className="form-label">
        {label}
      </label>
      <input type={type} id={id} name={id} className="form-input" placeholder={placeholder}
required={required} />
    </div>
  );
}
```

FICHIER `correction/src/components/molecules/Loading.jsx` — 11 lignes

```jsx
import { Spinner } from "../atoms";

// Le chargement pleine page : un spinner et un texte
export function Loading({ text = "Loading..." }) {
  return (
    <div className="center-page">
      <Spinner />
      <p>{text}</p>
    </div>
```

```
  );
}
```

FICHIER **correction/src/components/molecules/ErrorMessage.jsx** 8 lignes

```jsx
// Une erreur pleine page (ex. l'API ne répond pas)
export function ErrorMessage({ message }) {
  return (
    <div className="center-page">
      <p className="error-text">{message}</p>
    </div>
  );
}
```

FICHIER **correction/src/components/molecules/StatusMessage.jsx** 17 lignes

```jsx
import { AlertIcon, CheckIcon, Spinner } from "../atoms";

const ICONS = {
  loading: <Spinner />,
  success: <CheckIcon />,
  error: <AlertIcon />,
};

// Un encadré de message : variant = "loading" | "success" | "error"
export function StatusMessage({ variant, children }) {
  return (
    <div className={`message-box ${variant}-box`}>
      {ICONS[variant]}
      <span>{children}</span>
    </div>
  );
}
```

FICHIER **correction/src/features/favorites/hooks/useFavorites.js** 14 lignes

```jsx
import { useMemo } from "react";
import { useDispatch, useSelector } from "react-redux";
import { selectFavorites, toggleFavorite as toggleFavoriteAction } from "../favoritesSlice";

// Les écrans n'utilisent jamais useSelector / useDispatch directement : tout passe par ce hook
export const useFavorites = () => {
  const dispatch = useDispatch();
  const favorites = useSelector(selectFavorites);
  const favoriteIds = useMemo(() => favorites.map((car) => car.id), [favorites]);

  const toggleFavorite = (car) => dispatch(toggleFavoriteAction(car));

  return { favorites, favoriteIds, toggleFavorite };
};
```

FICHIER **correction/src/features/cars/hooks/useCars.js** 10 lignes

```jsx
import { useQuery } from "@tanstack/react-query";
import { fetchCarsByYear } from "../api/carsApi";
import { QUERY_KEYS } from "./queryKeys";

// L'année fait partie de la clé : chaque année a son propre cache
export const useCars = (year) =>
  useQuery({
    queryKey: [QUERY_KEYS.CARS_BY_YEAR, year],
    queryFn: () => fetchCarsByYear(year),
  });
```

FICHIER **correction/src/features/cars/hooks/useCarDetails.js** 11 lignes

```
import { useQuery } from "@tanstack/react-query";
import { fetchCarById } from "../api/carsApi";
import { QUERY_KEYS } from "./queryKeys";

// L'id fait partie de la clé : chaque voiture a son propre cache
export const useCarDetails = (id) =>
  useQuery({
    queryKey: [QUERY_KEYS.CAR_DETAILS, id],
    queryFn: () => fetchCarById(id),
    enabled: Boolean(id),
  });
```

FICHIER **correction/src/features/cars/adapters/carAdapter.js** 30 lignes

```
import { getColorHex } from "../utils/colorUtils";

// Texte affiché quand une valeur manque dans la réponse de l'API
export const MISSING_VALUE = "—";

const isMissing = (value) => value === undefined || value === null || value === "";
export const orMissing = (value) => (isMissing(value) ? MISSING_VALUE : value);

// Champs communs à la carte et à la page détails : DTO de l'API -> modèle pour l'UI
export const adaptCarBase = (dto = {}) => {
  const isAvailable = Boolean(dto.availability);
  return {
    id: dto.id,
    title: [dto.car, dto.car_model].filter((part) => !isMissing(part)).join(" ") || "Unknown car",
    colorHex: getColorHex(dto.car_color),
    colorName: orMissing(dto.car_color),
    year: orMissing(dto.car_model_year),
    vin: orMissing(dto.car_vin),
    priceLabel: orMissing(dto.price),
    isAvailable,
    statusLabel: isAvailable ? "Available" : "Unavailable",
  };
};

// Modèle d'une carte voiture (liste Home et Favoris)
export const adaptCarCard = (dto, isFavorite = false) => ({
  ...adaptCarBase(dto),
  detailsUrl: `/car/${dto?.id}`,
  isFavorite,
});
```

FICHIER **correction/src/components/organisms/index.js** 3 lignes

```
export { CarCard } from "./CarCard";
export { CarGrid } from "./CarGrid";
export { LoginForm } from "./LoginForm";
```

FICHIER **correction/src/components/atoms/Icons.jsx** 36 lignes

```
// Les icônes SVG, définies une seule fois et réutilisées partout

export function HeartIcon({ filled = false }) {
  return (
    <svg width="24" height="24" viewBox="0 0 24 24" fill={filled ? "currentColor" : "none"} stroke="currentColor" strokeWidth="2">
      <path d="M20.84 4.61a5.5 5.5 0 0 0-7.78 0L12 5.67l-1.06-1.06a5.5 5.5 0 0 0-7.78 7.78l1.06 1.06L12 21.23l7.78-7.78 1.06-1.06a5.5 5.5 0 0 0 0-7.78z" />
    </svg>
  );
}
```

```jsx

export function AlertIcon({ size = 24 }) {
  return (
    <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
      <circle cx="12" cy="12" r="10" />
      <line x1="12" y1="8" x2="12" y2="12" />
      <line x1="12" y1="16" x2="12.01" y2="16" />
    </svg>
  );
}

export function CheckIcon() {
  return (
    <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
      <path d="M22 11.08V12a10 10 0 1 1-5.93-9.14" />
      <polyline points="22 4 12 14.01 9 11.01" />
    </svg>
  );
}

export function BackIcon() {
  return (
    <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
      <path d="M19 12H5M12 19l-7-7 7-7" />
    </svg>
  );
}
```

FICHIER **correction/src/components/atoms/Button.jsx** 22 lignes

```jsx
import { Link } from "react-router-dom";

// Classes CSS d'un bouton : "btn", puis "btn-sm" et/ou "btn-success" / "btn-danger"
const buttonClassName = (size, variant) => ["btn", size && `btn-${size}`, variant && `btn-${variant}`].filter(Boolean).join(" ");

// Un vrai <button> (action)
export function Button({ size, variant, type = "button", children, ...props }) {
  return (
    <button type={type} className={buttonClassName(size, variant)} {...props}>
      {children}
    </button>
  );
}

// Un lien qui a l'apparence d'un bouton (navigation)
export function ButtonLink({ to, size, variant, children }) {
  return (
    <Link to={to} className={buttonClassName(size, variant)}>
      {children}
    </Link>
  );
}
```

FICHIER **correction/src/components/atoms/ColorDot.jsx** 4 lignes

```jsx
// La pastille de couleur : reçoit directement un code couleur (#rrggbb)
export function ColorDot({ color }) {
  return <div className="color-indicator" style={{ backgroundColor: color }}></div>;
}
```

FICHIER **correction/src/components/atoms/StatusBadge.jsx** 5 lignes

```jsx
// Disponible (vert) ou indisponible (rouge) ; className permet d'ajouter une classe (ex. "detail-value")
export function StatusBadge({ isAvailable, label, className }) {
  const statusClass = isAvailable ? "status-available" : "status-unavailable";
  return <span className={className ? `${className} ${statusClass}` : statusClass}>{label}</span>;
}
```

FICHIER **correction/src/components/atoms/Price.jsx** 4 lignes

```jsx
// Un prix déjà formaté (le formatage est fait par l'adapter)
export function Price({ value, className }) {
  return <span className={className ? `${className} price` : "price"}>{value}</span>;
}
```

FICHIER **correction/src/components/atoms/Spinner.jsx** 3 lignes

```jsx
export function Spinner() {
  return <div className="spinner"></div>;
}
```

FICHIER **correction/src/features/cars/api/carsApi.js** 7 lignes

```jsx
import axiosQuery from "@/lib/axiosQuery";

// Liste des voitures d'une année : l'API renvoie { Cars: [...] }
export const fetchCarsByYear = (year) => axiosQuery("get", `/api/cars/year/${year}`);

// Détails d'une voiture : l'API renvoie { Car: {...} }
export const fetchCarById = (id) => axiosQuery("get", `/api/cars/${id}`);
```

FICHIER **correction/src/features/cars/hooks/queryKeys.js** 5 lignes

```jsx
// Les clés React Query de la feature, centralisées pour éviter les fautes de frappe
export const QUERY_KEYS = {
  CARS_BY_YEAR: "cars",
  CAR_DETAILS: "car",
};
```

FICHIER **correction/src/features/cars/utils/colorUtils.js** 26 lignes

```jsx
// L'API donne un nom de couleur : on le transforme en code couleur pour la pastille.
const colorMap = {
  Red: "#e74c3c",
  Blue: "#3498db",
  Green: "#2ecc71",
  Yellow: "#f1c40f",
  Orange: "#f39c12",
  Purple: "#9b59b6",
  Pink: "#e91e63",
  Violet: "#9b59b6",
  Turquoise: "#1abc9c",
  Crimson: "#dc143c",
  Indigo: "#4b0082",
  Mauv: "#e0b0ff",
  Fuscia: "#ff00ff",
  Khaki: "#f0e68c",
  Maroon: "#800000",
  Aquamarine: "#7fffd4",
  Puce: "#cc8899",
  Goldenrod: "#daa520",
};

// Couleur inconnue ou absente : gris
export const DEFAULT_COLOR = "#95a5a6";
```

```
25
26 export const getColorHex = (colorName) => colorMap[colorName] || DEFAULT_COLOR;
```

FICHIER `correction/src/components/organisms/CarCard.jsx` — 29 lignes

```jsx
import { ButtonLink, ColorDot, Price, StatusBadge } from "../atoms";
import { FavoriteButton, InfoRow } from "../molecules";

// La carte d'une voiture : ne reçoit que des valeurs déjà prêtes à afficher (préparées par l'adapter)
export function CarCard({ title, colorHex, isAvailable, statusLabel, priceLabel, year, detailsUrl, isFavorite = false, onToggleFavorite }) {
  return (
    <div className="card car-card">
      <div className="card-header">
        <h2 className="car-name">{title}</h2>
        <FavoriteButton isActive={isFavorite} onToggle={onToggleFavorite} />
      </div>
      <div className="car-info">
        <InfoRow label="Color">
          <ColorDot color={colorHex} />
        </InfoRow>
        <InfoRow label="Status">
          <StatusBadge isAvailable={isAvailable} label={statusLabel} />
        </InfoRow>
        <InfoRow label="Price">
          <Price value={priceLabel} />
        </InfoRow>
        <InfoRow label="Year">
          <span>{year}</span>
        </InfoRow>
      </div>
      <ButtonLink to={detailsUrl}>View Details</ButtonLink>
    </div>
  );
}
```

FICHIER `correction/src/components/organisms/CarGrid.jsx` — 12 lignes

```jsx
import { CarCard } from "./CarCard";

// La grille de cartes ; onToggleFavorite reçoit l'id de la voiture cliquée
export function CarGrid({ cars, onToggleFavorite }) {
  return (
    <div className="car-grid">
      {cars.map((car) => (
        <CarCard key={car.id} {...car} onToggleFavorite={() => onToggleFavorite(car.id)} />
      ))}
    </div>
  );
}
```

FICHIER `correction/src/components/organisms/LoginForm.jsx` — 22 lignes

```jsx
import { Button } from "../atoms";
import { FormField } from "../molecules";

// Le formulaire de connexion : il lit les champs et renvoie { userName, password } au parent
export function LoginForm({ onSubmit, isPending, errorMessage }) {
  const handleSubmit = (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);
    onSubmit({ userName: formData.get("userName"), password: formData.get("password") });
  };

```

```
  return (
    <form onSubmit={handleSubmit} className="login-form">
      {errorMessage && <p className="error-text">{errorMessage}</p>}
      <FormField id="userName" label="Username" placeholder="Enter your username" required />
      <FormField id="password" label="Password" type="password" placeholder="Enter your password" required />
      <Button type="submit" disabled={isPending}>
        {isPending ? "Signing In..." : "Sign In"}
      </Button>
    </form>
  );
}
```

FICHIER **correction/src/lib/queryClient.js** 10 lignes

```
import { QueryClient } from "@tanstack/react-query";

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 3,
      refetchOnWindowFocus: true,
    },
  },
});
```

FICHIER **correction/src/lib/axiosQuery.js** 29 lignes

```
import axios from "axios";
import { BASE_URL } from "./config";
import { storage } from "@/features/auth/utils/storage";

const axiosInstance = axios.create({
  baseURL: BASE_URL,
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
  },
  timeout: 30000,
});

// Interceptor : on ajoute le token à chaque requête, s'il existe
axiosInstance.interceptors.request.use((config) => {
  const token = storage.getToken();
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Un seul point d'entrée pour tous les appels API : on récupère directement les données
const axiosQuery = async (method, url, data) => {
  const response = method === "post" ? await axiosInstance.post(url, data) : await axiosInstance.get(url);
  return response.data;
};

export default axiosQuery;
```

FICHIER **correction/src/lib/config.js** 2 lignes

```
// Les requêtes passent par le proxy Vite (/api) : pas besoin d'URL complète en dev
export const BASE_URL = "";
```

FICHIER **correction/index.html** 13 lignes

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/icon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Car Catalog</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

FICHIER **correction/vite.config.js** 26 lignes

```js
import { fileURLToPath, URL } from "node:url";
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  // Alias : "@/features/cars/..." au lieu de "../../../features/cars/..."
  resolve: {
    alias: {
      "@": fileURLToPath(new URL("./src", import.meta.url)),
    },
  },
  server: {
    // Le proxy évite les erreurs CORS en dev : /api/... est envoyé à https://myfakeapi.com/api/...
    proxy: {
      "/api/": {
        target: "https://myfakeapi.com",
        changeOrigin: true,
      },
    },
  },
  // Vitest : les tests unitaires (*.test.js) ; les tests e2e (Playwright) sont dans tests/e2e
  test: {
    include: ["src/**/*.test.js"],
  },
});
```